CONDITIONAL STATEMENTS

Control statements

A ***control statement*** is a statement that determines whether other statements will be executed. In most programming languages, the basic control statements include the following:
* **Conditional statements** (primarily the **if** statement)
* **Loop statements** (primarily the **for** and **while** statements)

The if statement

Conditional statements (the ***if statement*** and the ***switch statement***) execute or skip other statements on the value of a specified condition (aka *logical expression*). A ***logical expression*** (aka *Boolean expression*) is an expression that results in **`true`** or **`false`**.

Below are some examples of ***comparison operators*** that you can use in logical expressions.

`===`	`Equal to`	(equal value *and equal type*)
`!==`	`Not equal to`	(not equal value *and/or not equal type*)
`>`	`Greater than`	
`<`	`Less than`	
`>=`	`Greater than or equal to`	
`<=`	`Less than or equal to`	

The **if** statement allows JavaScript to execute statements conditionally. It has the following two formats:

```
if (logicalExpression)
    statement

if (logicalExpression)
    statement_1
else
    statement_2
```

The if statement syntax requires a single statement after the parenthezised logical expression. Here we use a statement block. A ***statement block*** allows you to use multiple statements where JavaScript syntax expects a single statement. A statement block is a sequence of statements enclosed within curly braces. The primitive statements in the statement block end with semicolons, but the statement block itself does not. The code is easier to read and understand when the lines inside the block are intended.

Example 1. A simple if statement

```
console.log("Your exam grade is " + grade);

if (grade === 0) {
    console.log("Please retake the exam.");
}
```

Example 2. An if statement with an else clause

```
if (age < 18) {
    console.log("Minor");
} else {
    console.log("Adult");
}
```

Example 3. Multiple choice (please notice the indentation)

```
if (age < 0) {
    console.log("The value is too low.");
} else if (age < 18) {
    console.log("Minor");
} else if (age < 65) {
    console.log("Adult");
} else if (age < 123) {
    console.log ("Senior");
} else {
    console.log ("The value is too high.");
}
```

Example 4. Multiple choice with the switch statement.

```
switch (grade) {
case 5:
    outputText = "Excellent";
    break;
case 4:
    outputText = "Very good";
    break;
case 3:
    outputText = "Good";
    break;
case 2:
    outputText = "Fair";
    break;
case 1:
    outputText = "Poor";
    break;
case 0:
    outputText = "Fail";
    break;
default:
    outputText = "Invalid grade number";
    break;
}
```

COMBINED CONDITIONS

Truth tables

Special party for club members only.

- Must have a membership card **AND** 10 euros for the entrance fee

Has a membership card?	Has 10 euros?	Allowed to enter?
true	false	
false	false	
false	true	
true	true	

Renting a bicycle

- Must have a credit card **OR** 100 euros for security deposit

Has a credit card?	Has 100 euros?	Can rent a bicycle?
false	false	
true	false	
false	true	
true	true	

Logical operators[1]

Logical operators allows you to combine multiple conditions into one logical expression. You can include any number of conditions in a single logical expression.

```
if (age < 0 || age > 122) {          // || means logical OR
    console.log("Invalid age.");
}

if (age >= 13 && age <= 19) {        // && means logical AND
    console.log("Teenager.");
}
```

JavaScript evaluates combined conditions ***from left to right***. But, you can use parenthesis to change (or sometimes to clarify) the order of evaluation.

[1] Simplified explonation only, the concept of truthy/falsy is not covered here.

The **&&** operator (AND) has higher precedence (it is 'stronger') than the **||** operator (OR).

```
if (age >= 65 || age >= 13 && age <= 19) {
    console.log ("Teenager or senior.");
}
```

IMPORTANT! In this type of situations, it is highly recommended to make the code more readable and understandable by clarifying the evaluation order with extra parenthesis.

```
if ((age >= 13 && age <= 19) || age >= 65) {
      console.log ("Teenager or senior.");
}
```

Normally, the **!** operator (logical NOT) is used to reverse the logical value of its operand. In the example below, the built-in JavaScript function returns ***true*** if the given value is ***not*** a proper number. The logical NOT operator is used here to reverse the logical value so that we can print a message when the temperature reading ***is*** a valid number.

```
if (!isNaN(temperatureReading)) {
    console.log ("The temperature value is a valid number.");
}
```

More advanced details

JavaScript "short-circuits" the evaluation of the logical operators as follows:

- If the left-hand side condition of the **||** operator is true, then the right-hand side condition of the **||** operator is not evaluated.
- If the left-hand side condition of the **&&** operator is false, then the right-hand side condition of the **&&** operator is not evaluated.

In the example below, we validate the value of the variable age with a compound condition. First, the **isNaN** function is used to test if the value is a number. If the value is not a number, then the sub condition evaluates to **true** and JavaScript ends the evaluation of the compound condition without evaluating the other sub conditions.

```
if (isNaN(age) || age < 0 || age > 122) {
    console.log ("Invalid age.");
}
```

A Complete JavaScript Program Example

AgePage.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8" />
    <title>Age Classification</title>
</head>
<body>
    <h1>Classify your age</h1>

    <form>
        <span>Your age (0 - 122): </span>
        <input type="text" id="txtAge" />
        <input type="button" value = "OK" onclick="classifyAge()" />
    </form>

    <p id="pOutput"></p>

    <script src="AgePage.js"></script>
</body>
</html>
```

AgePage.js

```
function classifyAge() {
    var outputText;

    var ageText = document.getElementById("txtAge").value;
    var age = Number(ageText);

    if (age < 18) {
        outputText = "You're a minor.";
    } else {
        outputText = "You're an adult.";
    }

    document.getElementById("pOutput").innerHTML = outputText;
}
```

AgePage.js (*** ADVANCED VERSION *** of the previous JavaScript code with input validation)

```
function classifyAge() {
    var outputText;

    var ageText = document.getElementById("txtAge").value;
    var age = Number(ageText);

    if (ageText.trim() === "" || isNaN(age)) {                          // A.
        outputText = "Please enter an integer between 0 and 122.";
    } else if (age < 0 || age > 122) {                                  // B.
        outputText = "The age should be between 0 and 122.";
    } else if (age < 18) {                                              // C.
        outputText = "You're a minor.";
    } else {
        outputText = "You're an adult.";                                // D.
    }

    document.getElementById("pOutput").innerHTML = outputText;
}
```

Comments

A. Here we validate the user input with a ***combined condition*** that includes two sub conditions. If either of the conditions below is true, then "*Please enter an integer between 0 and 122*." is assigned to the variable *outputText*.
   - the ***string is empty*** (the string does not contain any characters or it contains whitespace only)
   - the value of the variable ***age is not a number***

   The **trim** function removes whitespace from both ends of a string. **Whitespace** in this context is all non-visible characters. The **isNaN** function tests if the value is a number.

   JavaScript evaluates combined conditions from left to right. JavaScript "*short-circuits*" the evaluation of the OR-operator ("**||**"). That is, if the left-hand side condition of the OR-operator is true, then the right-hand side condition of the OR-operator is not evaluated.

B. This condition is tested only if all of the previous conditions are false. The condition takes care of too low (below 0) and too high (above 122) values. If the given age is too low or too high, then *"The age should be between 0 and 112.*" is assigned to the variable *outputText*.

C. This condition is tested only if all of the previous conditions are false. That is, we know for sure that the age to be tested here is a number between 0 and 122. **This condition is true if age is between 0 and 17.** "*You're a minor."* is assigned to the variable *outputText*.

D. If ***all of the previous conditions are false***, then JavaScript executes the code in this else clause. Condition B has taken care of ages that are below 0 and ages that are above 122. Condition C has taken care of ages that are between 0 and 17. What is left are ages between 18 and 122. "*You're an adult."* is assigned to the variable *outputText*.